

Evidencias Solicitadas

La ejecución del menú.

```
(venv) PS C:\Users\walmart_c\louthier\Documents\programacion2\proyectos\estructuraDatos> & "c:\Users\walmart_c\louthier\Documents\programacion2\proyectos\estructuraDatos\venv\Scripts\python.exe" "c:\Users\walmart_c\louthier\Documents\programacion2\proyectos\estructuraDatos/main_arbol.py"

ÁRBOL BINARIO DE BÚSQUEDA
1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir
Selecciona una opción: █
```

La inserción de los valores de la prueba inicial.

```
Selecciona una opción: 1
Ingresa el valor que deseas insertar: 50
Operación realizada.

ÁRBOL BINARIO DE BÚSQUEDA
1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir
Selecciona una opción: █
```

```
Selecciona una opción: 1
Ingresa el valor que deseas insertar: 30
Operación realizada.

ÁRBOL BINARIO DE BÚSQUEDA
1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir
Selecciona una opción: █
```

```
Selecciona una opción: 1
Ingresa el valor que deseas insertar: 70
Operación realizada.

ÁRBOL BINARIO DE BÚSQUEDA
1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir
Selecciona una opción: █
```

```
Selecciona una opción: 1
Ingresa el valor que deseas insertar: 20
Operación realizada.

ÁRBOL BINARIO DE BÚSQUEDA
1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir
Selecciona una opción: █
```

Explorador de archivos

```
Selecciona una opción: 1
Ingresa el valor que deseas insertar: 40
Operación realizada.
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

```
Selecciona una opción: █
```

```
Selecciona una opción: 1
Ingresa el valor que deseas insertar: 60
Operación realizada.
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

```
Selecciona una opción: █
```

```
Selecciona una opción: 1
Ingresa el valor que deseas insertar: 80
Operación realizada.
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

```
Selecciona una opción: █
```

Recorrido en preorden

```
Recorrido en preorden:
50 30 20 40 70 60 80
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

```
Selecciona una opción: █
```

Recorrido en Inorden

```
Selecciona una opción: 4
Recorrido en inorden:
20 30 40 50 60 70 80
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

```
Selecciona una opción: █
```

Recorrido en posorden

```
Selecciona una opción: 5  
Recorrido en posorden:  
20 40 30 60 80 70 50
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

Selecciona una opción: █

Una búsqueda exitosa.

```
Selecciona una opción: 2  
Ingresa el valor que deseas buscar: 20  
El valor se encuentra en el árbol.
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

Selecciona una opción: █

Una búsqueda de un valor inexistente.

```
Selecciona una opción: 2  
Ingresa el valor que deseas buscar: 2  
El valor no se encuentra en el árbol.
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

Selecciona una opción: █

El intento de insertar un valor duplicado.

```
Ingresa el valor que deseas insertar: 30  
El valor ya existe en el árbol.  
Operación realizada.
```

ÁRBOL BINARIO DE BÚSQUEDA

1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir

Selecciona una opción: █

El manejo de una entrada incorrecta.

```
Selecciona una opción: 1
Ingresa el valor que deseas insertar: Hola
Debes ingresar un número entero.
```

```
ÁRBOL BINARIO DE BÚSQUEDA
1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
7. Salir
Selecciona una opción: █
```

La ejecución del método contar.

```
(venv) PS C:\Users\walmart clouthier\Documents\programacion2\proyectos\estructuraDatos> & "c:\Users\walmart clouthier\Documents\pro"
gramacion2\proyectos\estructuraDatos\venv\Scripts\python.exe "c:/Users/walmart clouthier/Documents/programacion2/proyectos/estruct
uraDatos/main_arbol.py"
Selecciona una opción: 6
7

ÁRBOL BINARIO DE BÚSQUEDA
1. Insertar un valor
2. Buscar un valor
3. Recorrido en preorden
4. Recorrido en inorden
5. Recorrido en posorden
6. Contar nodo
```

El código de la clase BinaryTree.

```
class NodeTree:
```

```
    def __init__(self, value):
        self.value = value # valor almacenado
        self.left = None # se inician sin hijos
        self.right = None
```

```
class BinaryTree:
```

```
    def __init__(self):
        self.root = None # raíz del árbol
```

```
    def insertar(self, value):
        self.root = self._insertar(self.root, value) # Recibe solamente el valor con el que
se desea agregar
```

```
    def _insertar(self, node, value): # Permite desplazar por los nodos de todo el árbol
        if node is None: # Si el árbol está vacío, el valor de self.root será None.
```

```
        return NodeTree(value)

    if value < node.value:

        node.left = self._insertar(node.left, value)

    elif value > node.value:

        node.right = self._insertar(node.right, value)

    else:

        print("El valor ya existe en el árbol.") # NO SE CREA UN NODO DUPLICADO.

    return node
```

```
def buscar(self, value):

    return self._buscar(self.root, value)
```

```
def _buscar(self, node, value): # Comienza la búsqueda desde la raíz y se
desplaza por los nodos
```

```
    if node is None:

        return False # El Nodo no existe en el árbol

    if value == node.value:

        return True # El nodo es igual al que se está buscando

    if value < node.value:

        return self._buscar(node.left, value) # si es menor, va hacia la izquierda

    return self._buscar(node.right, value) # si es mayor va hacia la derecha
```

```
def preorden(self): # Raíz → Izquierda → Derecha

    self._preorden(self.root)

    print() # inicia el recorrido
```

```
def _preorden(self, node):
```

```
    if node is not None: # Cuando el método llega a una posición vacía, deja de realizar llamadas.
```

```
        print(node.value, end=" ") # Muestra el valor del nodo actual.
```

```
        self._preorden(node.left) # Recorre el subárbol izquierdo.
```

```
        self._preorden(node.right) # Recorre el subárbol derecho.
```

```
def inorden(self): # Izquierda → Raíz → Derecha
```

```
    self._inorden(self.root)
```

```
    print() # inicia el recorrido
```

```
def _inorden(self, node):
```

```
    if node is not None:
```

```
        self._inorden(node.left) # Recorre el subárbol izquierdo.
```

```
        print(node.value, end=" ") # Muestra el valor del nodo actual.
```

```
        self._inorden(node.right) # Recorre el subárbol derecho.
```

```
def posorden(self): # Izquierda → Derecha → Raíz
```

```
    self._posorden(self.root)
```

```
    print() # inicia el recorrido
```

```
def _posorden(self, node):
```

```
    if node is not None:
```

```
        self._posorden(node.left) # Recorre el subárbol izquierdo.
```

```
        self._posorden(node.right) # Recorre el subárbol derecho.
```

```
        print(node.value, end=" ") # Muestra el valor del nodo actual.
```

```
def contar(self):
```

```
"""
```

Regresa la cantidad total de nodos existentes en el árbol.

```
"""
```

```
return self._contar_recursivo(self.root)
```

```
def _contar_recursivo(self, nodo):
```

```
    # Si el nodo es None, debe contar cero.
```

```
    if nodo is None:
```

```
        return 0
```

```
    # Si existe un nodo, cuenta 1 + los nodos de la izquierda + los nodos de la derecha.
```

```
    return 1 + self._contar_recursivo(nodo.left) + self._contar_recursivo(nodo.right)
```

El código del menú principal.

```
from estructuras.no_lineales.binary_tree import BinaryTree
```

```
def mostrar_menu():
```

```
    print("\nÁRBOL BINARIO DE BÚSQUEDA")
```

```
    print("1. Insertar un valor")
```

```
    print("2. Buscar un valor")
```

```
    print("3. Recorrido en preorden")
```

```
    print("4. Recorrido en inorden")
```

```
    print("5. Recorrido en posorden")
```

```
    print("6. Contar nodo")
```

```
    print("7. Salir")
```

```
tree = BinaryTree()
```

```
while True:
```

```
    mostrar_menu()
```

```
    option = input("Selecciona una opción: ")
```

```
    if option == "1":
```

```
        try:
```

```
            value = int(input("Ingresa el valor que deseas insertar: "))
```

```
            tree.insertar(value)
```

```
            print("Operación realizada.")
```

```
        except ValueError:
```

```
            print("Debes ingresar un número entero.")
```

```
    elif option == "2":
```

```
        try:
```

```
            value = int(input("Ingresa el valor que deseas buscar: "))
```

```
            if tree.buscar(value):
```

```
                print("El valor se encuentra en el árbol.")
```

```
            else:
```

```
                print("El valor no se encuentra en el árbol.")
```

```
        except ValueError:
```

```
            print("Debes ingresar un número entero.")
```

```
    elif option == "3":
```

```
        print("Recorrido en preorden:")
```

```
        tree.preorden()
```



```
elif option == "4":
```

```
    print("Recorrido en inorden:")
```

```
    tree.inorden()
```

```
elif option == "5":
```

```
    print("Recorrido en posorden:")
```

```
    tree.posorden()
```

```
elif option == "6":
```

```
    print(tree.contar())
```

```
elif option == "7":
```

```
    print("Programa finalizado.")
```

```
    break
```

```
else:
```

```
    print("Opción no válida. Intenta nuevamente.")
```